

UC15 – Etapa 6

Relatório

Refatoração do sistema desktop ZeroAndar para reutilização na Web — Projeto Integrador 3

1. Contexto e objetivo

O sistema ZeroAndar foi desenvolvido originalmente como uma aplicação desktop em Java (PI2 – Etapa 4), utilizando Java Swing para a interface gráfica, controllers de acesso direto ao banco e DAOs concretos em JDBC/MySQL, contemplando o CRUD de Imóveis, Clientes, Proprietários e Corretores, além de tela de Login e Dashboard.

O enunciado da Etapa 6 solicitou a construção do núcleo reutilizável de um futuro sistema Web equivalente, a partir do reaproveitamento do código já desenvolvido. Como esse código estava fortemente acoplado ao Java Swing, foi necessário revisar e refatorar as classes, separando regra de negócio de interface, aplicando princípios SOLID e eliminando os code smells identificados.

Esta etapa não implementa ainda o back-end Web (Spring): o escopo é preparar o domínio, os contratos de persistência e os serviços de forma independente de qualquer framework de interface, deixando o projeto pronto para as Etapas 7 a 9.

2. Visão geral da refatoração

A refatoração manteve o mesmo domínio de negócio do sistema desktop (Pessoa, Cliente, Proprietário, Corretor, Imóvel, Endereço), porém reorganizou o código nos seguintes pacotes:

- model — entidades de domínio, sem qualquer dependência de Swing ou de banco de dados.
- repository — contratos (interfaces) de persistência, ex.: ClienteDAO, CorretorDAO, ImovelDAO, ProprietarioDAO.
- repository.jdbc — implementações JDBC/MySQL desses contratos, isolando o acesso ao banco.
- service — regras de negócio, validações e orquestração (ex.: ClienteService, ValidacaoService).
- infrastructure — configuração de acesso ao banco de dados (antiga DatabaseConnection).
- test — classe Main com os testes demonstrativos exigidos no enunciado.

As telas Swing (LoginFrame, DashboardFrame, ClientesPanel, CorretoresPanel, ImoveisPanel, ProprietariosPanel) e os componentes visuais customizados associados a elas foram excluídos do núcleo refatorado, já que não fazem sentido em um back-end Web e seriam substituídos por telas HTML/front-end nas próximas etapas.

3. Princípios SOLID aplicados

A tabela abaixo resume os princípios efetivamente aplicados, as classes impactadas e a justificativa de uso:

Princípio SOLID	Classes afetadas	Justificativa
SRP – Responsabilidade Única	Pessoa, PessoaCliente, PessoaProprietario, PessoaCorretor, Imovel,	No desktop, validações e regras ficavam misturadas em Controllers e nas próprias entidades. Na Etapa 6 as entidades passaram a

Princípio SOLID	Classes afetadas	Justificativa
	Endereco (model); ValidacaoService; ClienteService e demais services	conter apenas dados e comportamento essencial, as validações foram centralizadas em ValidacaoService e o acesso a dados foi isolado nos repositórios.
DIP – Inversão de Dependência	ClienteService, CorretorService, ImovelService, ProprietarioService	Os services dependem das interfaces ClienteDAO, CorretorDAO, ImovelDAO e ProprietarioDAO (pacote repository), nunca das implementações JDBC. Isso permitiu inclusive testar os services em memória (ClienteDAOEmMemoria), sem precisar de banco de dados.
ISP – Segregação de Interface	ClienteDAO, CorretorDAO, ImovelDAO, ProprietarioDAO	Cada entidade recebeu seu próprio contrato de persistência, em vez de uma interface genérica única, evitando que uma classe precise implementar métodos que não usa.
OCP – Aberto/Fechado	Pacote repository.jdbc (implementações) frente ao pacote repository (contratos)	Novas formas de persistência (ex.: JPA/Spring Data na Etapa 9) podem ser adicionadas criando novas implementações das interfaces, sem alterar o código já existente dos services.

4. Refatorações realizadas e padrão de projeto

Além dos princípios SOLID, as seguintes refatorações concretas foram aplicadas sobre o código herdado do projeto desktop:

Refatoração	Classes / métodos afetados	O que mudou e por quê
Remoção da camada Swing	LoginFrame, DashboardFrame, ClientesPanel, CorretoresPanel, ImoveisPanel, ProprietariosPanel (view)	Eliminadas do núcleo refatorado. Continham lógica de UI (JFrame/JPanel), componentes customizados (RoundedButton, RoundedPanel, RoundedTextField) e não fazem sentido em uma aplicação Web.
Controllers descontinuados / lógica extraída	ClientesController, CorretoresController, ImoveisController, ProprietariosController	No desktop, os Controllers instanciavam o DAO diretamente, tratavam SQLException e imprimiam feedback via System.out/err (acoplados à ideia de tela). Essa responsabilidade de orquestração de regra de negócio foi absorvida pelos novos Services.
DAOs transformados em contrato + implementação	ClienteDAO, CorretorDAO, ImovelDAO, ProprietarioDAO	Antes eram classes concretas com JDBC embutido. Foram divididas em uma interface (repository) e uma implementação JDBC (repository.jdbc), aplicando o padrão Repository.
Entidades de domínio mantidas e limpas	Pessoa, PessoaCliente, PessoaProprietario, PessoaCorretor, Endereco, Imovel, TelefonePessoa	Estrutura de herança (Pessoa como classe base) foi reaproveitada do projeto desktop; os modelos deixaram de ter qualquer referência a componentes Swing ou a conexão de banco.
Configuração de banco isolada	DatabaseConnection → infrastructure	Credenciais deixaram de ficar fixas no código; passaram a ser lidas de variáveis de ambiente (ZEROANDAR_DB_URL, ZEROANDAR_DB_USER, ZEROANDAR_DB_PASSWORD).

Padrão de projeto: Repository

Foi adotado o padrão Repository, separando o contrato de persistência (pacote repository) das implementações concretas em JDBC (pacote repository.jdbc). Essa escolha se justifica porque: (1) os services passam a depender apenas de abstrações, viabilizando testes sem banco de dados real; (2) a troca futura do JDBC por JPA/Spring Data (prevista para a Etapa 9) pode ser feita criando novas implementações, sem alterar o domínio nem os services; e (3) o padrão reflete diretamente os princípios DIP e OCP já citados na seção anterior.

5. Testes automatizados (classe Main)

O enunciado da Etapa 6 pede a implementação de testes no método main() para certificar que a refatoração está funcionando. Inicialmente foram elaborados 34 cenários de teste; entretanto, parte significativa deles dependia de conexão com banco de dados MySQL — recurso que a própria Etapa 6 não exige, já que o objetivo aqui é o núcleo desacoplado, não a persistência real.

Após a revisão, foram mantidos os 15 testes que efetivamente correspondem ao escopo da etapa: validações de negócio, integridade dos modelos de domínio e verificação da inversão de dependência (DIP) usando um repositório em memória (ClienteDAOEmMemoria), sem exigir MySQL. Essa é a versão presente na classe Main.java entregue.

Grupo de teste	Qtd.	Resultado
Validações (ValidacaoService) — 7 testes	7	15/15 aprovados
Modelos de domínio — 5 testes	5	15/15 aprovados
Inversão de dependência (DIP) — 3 testes	3	15/15 aprovados
TOTAL	15	15/15 (100%)

Execução registrada em log (mvn exec:java sobre uc15.etapa6.zeroandar.test.Main):

```
RESULTADO: 15/15 testes passaram.
BUILD SUCCESS
```

Conclusão da verificação: os 15 testes atuais estão corretamente enquadrados no enunciado da Etapa 6 (testes de regra de negócio e de desacoplamento, sem exigir banco de dados), sendo mais adequados do que o conjunto original de 34 cenários, que extrapolava o escopo desta etapa.

6. Evidências do repositório GitHub

O código-fonte completo do projeto, incluindo as etapas do UC15, está disponível no repositório público a seguir:

https://github.com/ale2k20/UC15_ZeroAndar_Final

7. Conclusão

A refatoração atingiu o objetivo da Etapa 6: o núcleo de domínio e regra de negócio do sistema ZeroAndar foi desacoplado do Java Swing, reorganizado em camadas (model, repository, service, infrastructure) e validado

por 15 testes automatizados executados via `main()`, todos aprovados. O projeto está preparado para receber, nas próximas etapas, a camada Web (controllers REST/MVC, views HTML) e a futura migração de JDBC para JPA/Spring Data, sem necessidade de alterar as regras de negócio já implementadas.